BLUEPRINT · REV 1.0 · FINAL YEAR PROJECT EDITION

# NeuroForge
# AI-First Enterprise SDLC Platform

A curated 15-module architecture blueprint benchmarked against what Jira, Azure DevOps, GitLab Duo and modern AI-SDLC workspaces actually ship in 2026 — scoped so one student team can build it in an academic year.

| 15 | 6 | 8 | 7 |
|---|---|---|---|
| MODULES | LOGIN ROLES | MONTH PLAN | AI FEATURES |

STACK — **Java 17 + Spring Boot 3** · **React JS + Tailwind** · **PostgreSQL** (relational core) · **MongoDB** (AI docs, logs, audit) · **LLM API** · **WebSockets**

PART A

## User Logins & Role Matrix

Six login roles, mirroring how Jira and Azure DevOps structure access. Every endpoint and every UI element is gated by role — a Developer never even sees the "Create Project" button.

| ROLE | LOGIN TYPE | WHAT THEY CAN DO |
|---|---|---|
| **Super Admin** | Platform-level | Manages the entire platform: creates organizations, monitors usage, manages plans, views platform analytics. This is you as the platform owner. |
| **Org Admin** | Organization-level | Manages one company: creates teams, invites members, assigns roles, configures org settings, views all projects in their org. |

| | | |
|---|---|---|
| **Project Manager** | Member | Creates projects, defines requirements, plans sprints, assigns tasks, approves specs, manages releases, views all reports. |
| **Developer** | Member | Views assigned tasks, updates status, submits code for AI review, links commits, logs work, raises blockers. |
| **QA / Tester** | Member | Creates & executes test cases, reports bugs, verifies fixes, generates AI test cases, manages test runs. |
| **Client / Stakeholder** | Viewer · read-only | Views project progress, milestones, release notes and approved documents. Cannot edit anything. |

### Login Flow Architecture

1. User submits email + password → React sends to `POST /api/auth/login`
2. Spring Security validates credentials → generates JWT with role claims
3. JWT stored in frontend (httpOnly cookie) → sent with every API call
4. Backend filters check role on every endpoint (only PM can create a sprint)
5. React reads role from JWT to show or hide UI per role

PART B

# The 15 Modules

Each module is a spec sheet: what it does, the real platform it's benchmarked against, who logs into it, the architecture flow, where data lives, and how to build it.

ALL 15 | AI-POWERED | CORE SDLC | REAL-TIME

## 01 Authentication, RBAC & User Management

BENCHMARKED AGAINST — Jira & Azure DevOps role-based access

CORE

— WHAT IT DOES

The security foundation. Handles signup, login, JWT sessions, password reset via email OTP, and the 6-role permission system. Every other module depends on this.

— WHO LOGS IN

All 6 roles — it *is* the login system

— ARCHITECTURE FLOW

```
React Login Page → POST /api/auth/login → Spring Security Filter Chain
→ UserDetailsService loads user from PostgreSQL → BCrypt password check
→ JWT generated (userId, orgId, role) → returned to React
→ every request: JWT in header → JwtAuthFilter → role check → controller
```

POSTGRESQL

users · roles · permissions · org_memberships · password_reset_tokens

**Build:** Week 1 — schema + Spring Security + JWT util. Week 2 — auth APIs + React pages. Week 3 — role guards in React + `@PreAuthorize` rules.

## 02 Organization & Team Workspace

BENCHMARKED AGAINST — Azure DevOps Organizations → Projects hierarchy · Jira workspaces

CORE

— WHAT IT DOES

Multi-tenant structure. One platform hosts many organizations; each org has teams; each team has members with roles. Org Admin sends email invites; users join via invite link.

— WHO LOGS IN

SA Creates orgs | OA Manages teams & invites | ALL Belong to teams

— ARCHITECTURE FLOW

```
Org Admin → "Invite Member" → POST /api/orgs/{id}/invites
→ unique invite token saved → email sent (JavaMailSender)
→ invitee clicks link → accept page → membership row inserted
→ EVERY query scoped: WHERE org_id = {jwt.orgId}  ← the multi-tenancy rule
```

POSTGRESQL

organizations · teams · team_members · invites

**Build:** Immediately after Module 1. Test with 2 fake orgs and verify data never leaks between them. The *org_id-on-every-table* rule is what makes this "enterprise".

## 03 Project & Portfolio Management

BENCHMARKED AGAINST — Jira Projects · monday dev roadmap hierarchy

CORE

— WHAT IT DOES

The parent container. PM creates a project with methodology (Agile/Waterfall), dates, tech-stack tags and a team. Portfolio view shows all projects with health indicators: On Track · At Risk · Delayed.

— WHO LOGS IN

PM Create / edit | OA View all | DEV·QA View assigned | CL Progress only

— ARCHITECTURE FLOW

```
PM → Create Project wizard (React multi-step form)
→ POST /api/projects → ProjectService validates team → saves
→ nightly @Scheduled job: % tasks done vs % timeline elapsed → health flag
→ Portfolio: GET /api/projects?orgId=X → cards with health badges
```

POSTGRESQL

projects · project_members · milestones · project_health_snapshots

**Build:** Simple CRUD first; add the health-calculation job later. Even a simple formula makes the portfolio look intelligent.

## 04 AI Requirements & Spec Studio

BENCHMARKED AGAINST — **AWS Kiro spec-driven development · "specs as versioned artifacts" trend**

AI-POWERED

— WHAT IT DOES

PM writes a plain-English feature description; AI converts it into user stories, acceptance criteria and functional/non-functional requirements. Every spec is versioned (v1, v2, v3...) with an approval workflow: Draft → In Review → Approved. Approved specs become the source that tasks and test cases trace back to.

— WHO LOGS IN

- **PM** Writes + approves
- **DEV** Reads approved specs
- **QA** Generates tests from specs
- **CL** Views approved versions

— ARCHITECTURE FLOW

```
PM types: "Users should be able to book a CNG slot and pay online"
→ POST /api/specs/generate → LLM prompt: "convert to user stories + criteria, return JSON"
→ JSON parsed → editable React form → PM edits → saved as v1
→ edits after approval create a NEW version (old ones immutable)
→ Approve click → status change → team notified
→ spec content in MongoDB · metadata + versions in PostgreSQL
```

POSTGRESQL + MONGODB

spec metadata, versions, approval status → SQL · full spec JSON → Mongo

> **Build:** Your first AI module — month 3–4, once CRUD is stable. The versioning + approval workflow is what examiners will love.

## 05 Agile Planning: Boards, Sprints & Backlog

BENCHMARKED AGAINST — **Jira Scrum/Kanban · Azure Boards**

CORE REAL-TIME

— WHAT IT DOES

The daily driver. Backlog → sprint planning → Kanban board (To Do / In Progress / Code Review / Testing / Done) → auto sprint summary. Tasks carry story points, priority, assignee, labels, and link back to a spec requirement for traceability.

— WHO LOGS IN

PM Plans sprints, assigns | DEV·QA Move own tasks | CL Board read-only

— ARCHITECTURE FLOW

```
Kanban drag-and-drop (dnd-kit)
→ on drop: PATCH /api/tasks/{id}/status → optimistic UI update
→ backend validates transition rules (only QA can move to "Done")
→ status event → WebSocket broadcast → everyone's board updates live
→ burndown: nightly snapshot of remaining points → Recharts line graph
```

POSTGRESQL

sprints · tasks · task_status_history · story_point_snapshots

**Build:** Backlog CRUD → board UI → drag-drop → sprint lifecycle → burndown → WebSocket sync, in that order. Live board sync is the feature that makes the demo feel like a real product.

06

## AI Ticket Triage & Smart Assignment

BENCHMARKED AGAINST — Jira automations (auto-triage / auto-assign) · monday dev bug categorization

AI-POWERED

— WHAT IT DOES

A new ticket arrives with just a title and description. AI suggests: category (Frontend/Backend/DB/DevOps), priority, estimated story points, and best assignee based on who handled similar tickets before. PM accepts with one click or overrides.

— WHO LOGS IN

PM Accepts / overrides suggestions | DEV·QA Benefit from correct routing

— ARCHITECTURE FLOW

```
new ticket → TicketService fires async event (@Async)
→ TriageService builds prompt: ticket text + team members + past categories
→ LLM returns {category, priority, points, suggestedAssignee, reasoning}
→ saved to MongoDB → "AI Suggests" panel on the ticket
→ PM clicks Accept → fields applied → assignment notification
```

MONGODB + POSTGRESQL

ai_triage_suggestions (with reasoning) → Mongo · applied values → SQL

**Build:** Reuses Module 4's LLM plumbing — high wow-factor for low effort. Displaying the *reasoning* ("suggested Kumanan — resolved 8 similar payment bugs") is the demo highlight.

## 07 Repository & Commit Integration

BENCHMARKED AGAINST — **Jira ↔ GitHub integration · GitLab Orbit lifecycle graph**

CORE

— WHAT IT DOES

Connect a GitHub repo via the free REST API. The platform syncs branches, commits and PRs, and auto-links them to tasks through commit-message conventions ("NF-123" in the message → linked to task NF-123). Every task page shows its related commits — requirement → task → code traceability.

— WHO LOGS IN

PM Connects repo, views traceability

DEV Commits auto-link

— ARCHITECTURE FLOW

```
PM enters repo URL + token → saved encrypted
→ scheduled sync (10 min) or "Sync Now" → GitHub REST API → new commits/PRs
→ regex scans messages for NF-\d+ → task_commit_links created
→ task page: GET /api/tasks/{id}/commits → commit list + diff links
```

POSTGRESQL

repo_connections · commits_cache · task_commit_links

**Build:** Polling is simpler than webhooks for a student project — mention webhooks as a future enhancement in your report.

## 08 AI Code Review Assistant

BENCHMARKED AGAINST — GitHub Copilot code review · GitLab Duo suggestions

AI-POWERED

### — WHAT IT DOES

Developer pastes code or picks a synced commit; AI returns structured feedback — bugs, security issues, performance suggestions and a 1–10 quality score. Review history builds quality trends per developer. A human still gives final approval, mirroring the real-world "AI reviews + human validates" pattern.

### — WHO LOGS IN

DEV Requests review

PM Quality trends + final approval

### — ARCHITECTURE FLOW

```
paste code / pick commit → POST /api/reviews/analyze
→ ReviewService chunks large code → LLM with fixed review rubric prompt
→ {issues:[{line, severity, description, suggestion}], score, summary}
→ MongoDB doc → React renders severity color badges
→ workflow gate: task can't reach Testing until review Accepted
```

MONGODB + POSTGRESQL

code_reviews (full issue arrays) → Mongo · review status on task → SQL

**Build:** Your strongest AI differentiator. A fixed rubric (correctness, security, performance, readability) keeps output consistent; the workflow gate shows enterprise governance thinking.

## 09 CI/CD Pipeline & Release Tracker

BENCHMARKED AGAINST — Azure Pipelines · GitLab CI/CD · Jenkins (simulated for scope)

CORE REAL-TIME

— WHAT IT DOES

Visual pipeline: Build → Unit Test → Security Scan → Deploy Dev → Deploy QA → Deploy Prod. Runs are simulated with realistic timing and configurable pass rates — the architecture is identical to real CI/CD dashboards. Releases group tasks into versions with AI-generated release notes.

— WHO LOGS IN

PM Triggers releases, approves prod | DEV Views build status | CL Reads release notes

— ARCHITECTURE FLOW

```
PM → "Run Pipeline" → POST /api/pipelines/run
→ PipelineSimulator (@Async) walks stages with delays
→ each stage update → WebSocket push → UI animates grey → blue → green/red
→ prod stage requires PM approval click (approval gate)
→ release notes: LLM summarizes all Done tasks → editable draft
```

POSTGRESQL

pipelines · pipeline_runs · pipeline_stages · releases · release_tasks

**Build:** Simulation keeps scope sane; the live-animating pipeline is a spectacular demo moment. Stretch goal: trigger one real GitHub Actions workflow via API.

## 10 Test Management & AI Test Case Generation

BENCHMARKED AGAINST — Azure Test Plans + industry-standard AI test generation

CORE AI-POWERED

— WHAT IT DOES

QA creates test cases mapped to spec requirements; test runs record pass/fail/blocked per build. AI feature: pick an approved requirement → AI generates positive, negative and edge-case tests → QA reviews and imports. A coverage report highlights requirements with no tests.

— WHO LOGS IN

QA Full access | PM Views coverage | DEV Sees failed tests on their tasks

— ARCHITECTURE FLOW

```
QA selects requirement → POST /api/testcases/generate
→ LLM receives requirement + acceptance criteria → test case array JSON
→ QA reviews selectable list → imports → PostgreSQL
→ test run → mark results → failed case auto-creates a bug (Module 11)
→ coverage: SQL join requirements ↔ test_cases → untested shown in red
```

POSTGRESQL

test_cases · test_steps · test_runs · test_results

**Build:** The "failed test auto-creates a bug" automation is the key integration to demo — modules talking to each other is what makes this a *platform*.

## 11 Bug Tracking & Incident Management

BENCHMARKED AGAINST — **Jira issue tracking · Azure SRE agents (incident troubleshooting)**

CORE REAL-TIME

— WHAT IT DOES

Full lifecycle: Open → Triaged → In Progress → Fixed → Verified → Closed, with severity, environment, attachments, and AI duplicate detection ("possible duplicate of BUG-42"). Critical bugs trigger incident mode: team-wide alert banner and an SLA timer counting time-to-resolution.

— WHO LOGS IN

QA Reports / verifies | DEV Fixes | PM Triages, monitors SLA | CL Views critical incident status

— ARCHITECTURE FLOW

```
bug submitted → async duplicate check:
   LLM compares title+description similarity vs open bugs → warning shown
→ severity = Critical → IncidentService:
   WebSocket broadcast + email all members + SLA timer started
→ bug board reuses Kanban components from Module 5
→ "Verified" transition allowed only for QA role
```

POSTGRESQL + MONGODB

bugs · status_history · incidents · sla_timers → SQL · duplicate-check logs → Mongo

**Build:** Reuse Module 5's board heavily. The visible SLA countdown on critical bugs is a small feature with big enterprise credibility.

## 12 DevSecOps: Security & Compliance Scanning

BENCHMARKED AGAINST — **GitLab built-in security scanning · shift-left DevSecOps**

CORE AI-POWERED

### — WHAT IT DOES

Two scanners. Dependency scan: upload a *package.json* or *pom.xml* and check every dependency against known vulnerabilities via the free OSV.dev API. AI code scan: pasted code checked for hardcoded secrets, SQL-injection patterns and risky practices. Findings convert to tickets in the bug module.

### — WHO LOGS IN

DEV Runs scans | PM Security posture dashboard | OA Compliance overview

### — ARCHITECTURE FLOW

```
upload pom.xml → POST /api/security/scan-deps
→ parse dependencies → batch query OSV.dev API → vulnerabilities mapped
→ findings severity-scored → "Create ticket" → bug (Module 11)
→ dashboard: open findings by severity · trend · per-project risk score
```

POSTGRESQL

security_scans · findings · finding_ticket_links

**Build:** OSV.dev is free with no API key — perfect for students. Dependency scan first; the AI secret scan is a quick add-on reusing LLM plumbing. "DevSecOps" is a term that impresses any panel.

## 13 Traceability & Audit Compliance Engine

BENCHMARKED AGAINST — **2026 AI-SDLC workspaces (LoopIQ): roadmap → PR → deployment traceability, automated audit evidence**

CORE

— WHAT IT DOES

The connective tissue. One click on any requirement shows the full chain: Requirement v2 → Task NF-123 → Commits (3) → Code Review (score 8) → Tests (5, all passed) → Release v1.2.0 → Deployed to Prod by user Y. Plus an immutable audit log of every significant action, exportable as a PDF report.

— WHO LOGS IN

OA·PM Audits, compliance reports

CL Trust / verification view

— ARCHITECTURE FLOW

```
every service publishes events → AuditService (ApplicationEventPublisher)
→ events appended to MongoDB (immutable — never updated or deleted)
→ trace chain: backend walks FK links across modules
   requirement → tasks → commits → reviews → test_results → release
→ GET /api/trace/requirement/{id} → visual chain diagram in React
→ "Export Audit Report" → PDF (OpenPDF)
```

MONGODB + POSTGRESQL

audit_events (append-only) → Mongo · existing FK links do the traceability work

**Build:** Build LAST — it needs every other module's data. Little code (it mostly reads links), but it transforms 14 features into one coherent platform. Make the chain diagram your demo climax.

## 14 Analytics & Delivery Intelligence Dashboard

BENCHMARKED AGAINST — **monday dev AI sprint snapshots · Jira reports · DORA metrics**

CORE AI-POWERED

— WHAT IT DOES

Role-aware dashboards. PM: velocity trend, burndown, cycle time, bug trends, DORA-style deployment frequency and change-failure rate. Org Admin: cross-project portfolio health. Developer: personal

stats. One button generates an AI "Sprint Health Summary" in plain language with risks and recommendations.

— WHO LOGS IN

All roles — each sees their own variant

CL Simplified progress view

— ARCHITECTURE FLOW

```
nightly @Scheduled aggregation → metrics_snapshots
   (pre-aggregation = fast dashboards — real enterprise pattern)
→ GET /api/analytics/{scope} → snapshot JSON → Recharts
→ "AI Summary": sprint stats → LLM → executive summary text
→ PDF export for stakeholder reports
```

POSTGRESQL

metrics_snapshots · velocity_history (computed from existing data)

**Build:** Start with 3 charts (velocity, burndown, bug trend) and grow. The AI sprint summary reuses LLM plumbing and lands management-level wow factor.

## 15 NeuroBot: AI Assistant & Notification Hub

BENCHMARKED AGAINST — GitLab Orbit queryable lifecycle graph · Atlassian Rovo agents

AI-POWERED REAL-TIME

— WHAT IT DOES

Two halves. NeuroBot chat answers questions about *your* data — "What's pending in Sprint 3?", "Which bugs are critical?" — via a mini-RAG pattern over your own database. The notification hub delivers real-time WebSocket alerts plus email digests for assignments, approvals, incidents and mentions, with per-user preferences.

— WHO LOGS IN

All roles — answers respect permissions

CL Asking about internal bugs gets a polite refusal

— ARCHITECTURE FLOW

```
"what's pending in sprint 3?" → POST /api/bot/ask
→ step 1: LLM classifies intent → {intent: TASK_QUERY, sprint: 3}
```

```
→ step 2: BotService runs a SAFE predefined query
   (the LLM NEVER writes raw SQL — this is the security design)
→ step 3: results + question → LLM → natural language answer
notifications: domain events → WebSocket topic per user + email queue
```

MONGODB + POSTGRESQL

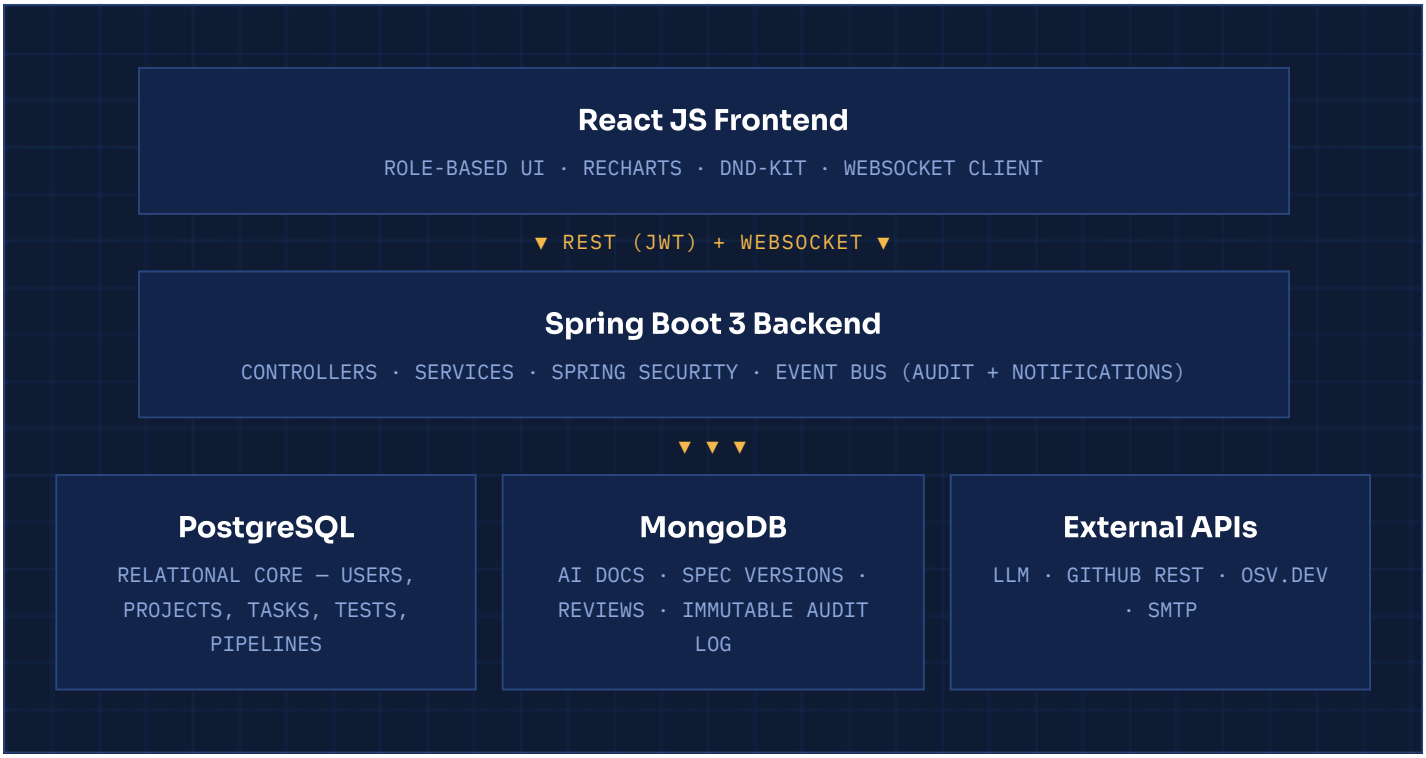
bot conversations → Mongo · notifications + preferences → SQL

**Build:** Build notifications early (other modules need them); build the bot near the end when data exists. Explaining the intent→safe-query→answer pattern in your viva demonstrates real architectural maturity.

PART C

# How It All Connects

One frontend, one backend, two databases with clearly divided responsibilities, and external APIs for AI, code and security intelligence.




PART D

# 8-Month Build Plan

Six phases, each ending in something demonstrable. CRUD foundations first, AI differentiators in the middle, the traceability engine last — because it needs everyone else's data to exist.

Months 1–2

### Foundation

MODULES 01 · 02 · 03

Multi-tenant platform with logins, organizations and projects working end to end.

Months 3–4

### Core SDLC

MODULES 05 · 11 · 15 (notifications)

Boards, sprints and bugs — a usable Jira-lite your team can actually run the rest of the project inside.

Month 5

### AI Layer

MODULES 04 · 06 · 08

Spec studio, AI triage and AI code review — the differentiators.

Month 6

### Delivery

MODULES 07 · 09 · 10

Repo integration, animated pipeline tracker and test management.

Month 7

### Intelligence

MODULES 12 · 14 · 15 (bot)

Security scans, dashboards and NeuroBot.

Month 8

### Enterprise Polish

MODULE 13

Traceability engine, audit reports and final demo preparation.

### Viva / Demo Strategy — three moments, maximum impact

1. Open with the traceability chain: one click showing requirement → code → test → deployment proves everything is connected. 2. Show one live AI moment — ticket triage or code review with visible reasoning. 3. Close with the pipeline animating live over WebSockets. Then take questions on the two design decisions that show maturity: the **org_id multi-tenancy rule** and the **NeuroBot safe-query pattern** where the LLM never writes raw SQL.